

```
import re
import requests
import string
from bs4 import BeautifulSoup
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
```

```
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('punkt_tab')
nltk.download('wordnet')
nltk.download('omw-1.4')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
[nltk_data] Package omw-1.4 is already up-to-date!
True
```

ดึงข้อความจาก url

```
url = 'https://stackoverflow.com/questions/38488442/razor-framework-backend-or-frontend'
page = requests.get(url)
soup = BeautifulSoup(page.text, 'html')
```

Web Scraping

```
import requests

# Question ID
question_id = 38488442

# --- Fetch the question ---
question_url = f'https://api.stackexchange.com/2.3/questions/{question_id}'
question_params = {
    'order': 'desc',
    'sort': 'activity',
    'site': 'stackoverflow',
    'filter': 'withbody'
}

q_response = requests.get(question_url, params=question_params)
q_data = q_response.json()

if 'items' in q_data and q_data['items']:
    question = q_data['items'][0]
    print("=== Question ===")
    print("Title:", question['title'])
    print("Body:", question['body'])
    print("Score:", question['score'])
    print("Asked by:", question['owner'].get('display_name', 'Unknown'))
    print(f"***50, *\n")
else:
    print("Question not found")

# --- Fetch the answers ---
answer_url = f'https://api.stackexchange.com/2.3/questions/{question_id}/answers'
answer_params = {
    'order': 'desc',
    'sort': 'votes',
    'site': 'stackoverflow',
    'filter': 'withbody'
}

a_response = requests.get(answer_url, params=answer_params)
a_data = a_response.json()

answers = a_data.get('items', [])
if answers:
    for i, ans in enumerate(answers, start=1):
        print(f"--- Answer {i} ---")
```

```

print("Body:", ans['body'])
print("Score:", ans['score'])
print("Answered by:", ans['owner'].get('display_name', 'Unknown'))
print("-"*50, "\n")
else:
    print("No answers found")

```

Body: <p>What kind of framework is Razor? Is it backend or frontend?  
What is the difference between the two types of frameworks?  
I'm trying to learn a little bit more about backend and frontend frameworks and since I usually work with Visual Studio Asp.n

Score: 7  
Asked by: Diogo Martinho  
=====

--- Answer 1 ---  
Body: <p>It is not a framework . I think you're misinterpreting certain concepts. Razor is a server side view engine, and it  
<p><a href="http://haacked.com/archive/2011/01/06/razor-syntax-quick-reference.aspx/" rel="noreferrer">Razor Syntax Quick Ref

Score: 6  
Answered by: Eulogy  
-----

--- Answer 2 ---  
Body: <p>This question is a couple of years old, but I'm going to add my two cents.</p>  
<p>People struggle to give an answer to this question because the terms 'front-end' and 'back-end' aren't formally defined an  
<p>That being said, it is my <em>opinion</em> that the relationship between front-end/back-end and client-side/server-side is  
<p>I think it helps to think of it like this: <strong>client-side and server-side are run-times</strong>, while <strong>front  
<p>'Client-side' always refers to code executing on the client's machine and 'server-side' always refers to code executing on  
<p>Consider a front-end developer who is tasked with building a UI. Much of the code they write will be the typical HTML / CS  
<p>That is, <strong>the front-end developer will write server-side code to help generate the UI, in addition to writing the c  
<p>Now, I can't imagine a scenario where a back-end developer will write client-side code.</p>  
<p>So, to answer your question, <strong>Razor is a front-end technology that executes on the server-side runtime</strong>. It

Score: 5  
Answered by: koolahman  
-----

--- Answer 3 ---  
Body: <p>Razor is for writing dynamic html page which is front end and c# is for writing backend logic. Although you could mo

Score: 1  
Answered by: Steve  
-----

--- Answer 4 ---  
Body: <p>Razor allows you on the <strong>back-end</strong> more easily create views (.cshtml in C#).  
It is more like templating system...</p>  
<p><a href="http://www.asp.net/web-pages/overview/getting-started/introducing-razor-syntax-c" rel="nofollow">http://www.asp.n

Score: 1  
Answered by: alex.dev  
-----

### Question Part

```

div_tags = soup.find_all('div', class_='s-prose js-post-body', attrs={'itemprop': 'text'})
question = div_tags[0].text.strip('\n')
print(question)

```

What kind of framework is Razor? Is it backend or frontend?  
What is the difference between the two types of frameworks?  
I'm trying to learn a little bit more about backend and frontend frameworks and since I usually work with Visual Studio Asp.net

### 1.cleaning

```

#clean_punctuation
def clean_punctuation(text):
    punctuation_pattern = re.compile(r'[^w\s]|_')
    cleaned_text = re.sub(punctuation_pattern, '', text)

```

```

        return cleaned_text

question = clean_punctuation(question)
question = question.replace('\n', ' ')
question

```

'What kind of framework is Razor Is it backend or frontend What is the difference between the two types of frameworks Im tryin  
g to learn a little bit more about backend and frontend frameworks and since I usually work with Visual Studio Aspnet MVC was

```

# lower character
question = question.lower()
print(question)

```

what kind of framework is razor is it backend or frontend what is the difference between the two types of frameworks im trying

```

# remove stopwords
words = word_tokenize(question)

stop_words = set(stopwords.words('english'))
filtered_words = [word for word in words if word.lower() not in stop_words]
question_cleaning_text = " ".join(filtered_words)
print(question_cleaning_text)

```

kind framework razor backend frontend difference two types frameworks im trying learn little bit backend frontend frameworks si

```

# remove special characters
question_cleaning_text = re.sub(r'^a-zA-Z0-9\s', '', question_cleaning_text)
question_cleaning_text = re.sub(r'\s+', ' ', question_cleaning_text).strip()

print(question_cleaning_text)

```

kind framework razor backend frontend difference two types frameworks im trying learn little bit backend frontend frameworks si

```

# Data masking
def masking(text):
    prog_pattern = re.compile(r'Razor|Aspnet')
    masked_text = re.sub(prog_pattern, '[FRAMEWORK]', text)

    return masked_text
question_cleaning_text = masking(question_cleaning_text)
print(question_cleaning_text)

```

kind framework razor backend frontend difference two types frameworks im trying learn little bit backend frontend frameworks si

## 2. Pre-Processing

```

# Lemmatizer
lemmatizer = WordNetLemmatizer()
words = question_cleaning_text.split(' ')
question_preprocessing_text = ' '.join([lemmatizer.lemmatize(word, pos='n') for word in words])
question_preprocessing_text = ' '.join([lemmatizer.lemmatize(word, pos='v') for word in words])
print(question_preprocessing_text)

```

kind framework razor backend frontend difference two type frameworks im try learn little bite backend frontend frameworks since

```

# POS-tag
from nltk import pos_tag
nltk.download('averaged_perceptron_tagger')
tokens_for_pos = nltk.word_tokenize(question_preprocessing_text)
pos_tags = pos_tag(tokens_for_pos)

print(pos_tags)

```

```

[('kind', 'NN'), ('framework', 'NN'), ('razor', 'NN'), ('backend', 'VBP'), ('frontend', 'JJ'), ('difference', 'NN'), ('two', 'C
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]   /root/nltk_data...
[nltk_data]   Package averaged_perceptron_tagger is already up-to-
[nltk_data]   date!

```

### Answer Part

```

ans_zone = soup.find_all('div', class_='answer-cell post-layout--right')
answers = [elem.find_all('p') for elem in ans_zone]

```

answers

```
[[<p>It is not a framework . I think you're misinterpreting certain concepts. Razor is a server side view engine, and it uses C # or VB.NET to generate dynamic content.</p>,
  <p><a href="http://haacked.com/archive/2011/01/06/razor-syntax-quick-reference.aspx/" rel="noreferrer">Razor Syntax Quick Reference</a></p>],
  <p>This question is a couple of years old, but I'm going to add my two cents.</p>,
  <p>People struggle to give an answer to this question because the terms 'front-end' and 'back-end' aren't formally defined anywhere. Because of that, any answer is purely subjective.</p>,
  <p>That being said, it is my <em>opinion</em> that the relationship between front-end/back-end and client-side/server-side isn't necessarily one-to-one</p>,
  <p>I think it helps to think of it like this: <strong>client-side and server-side are run-times</strong>, while <strong>front-end and back-end are a separation of concerns</strong>.</p>,
  <p>'Client-side' always refers to code executing on the client's machine and 'server-side' always refers to code executing on the server. A 'front-end' developer deals with displaying data to the user and getting data from the user, while a back-end developer deals with storing, manipulating, and retrieving that data.</p>,
  <p>Consider a front-end developer who is tasked with building a UI. Much of the code they write will be the typical HTML / CSS / JS. However, they will also have to deal with the data that is passed to the front-end from the backend. This is where Razor comes into play. The front-end developer will write the Razor code (which executes on the server-side) to display the data. </p>,
  <p>That is, <strong>the front-end developer will write server-side code to help generate the UI, in addition to writing the client-side code that really defines the UI</strong>.</p>,
  <p>Now, I can't imagine a scenario where a back-end developer will write client-side code.</p>,
  <p>So, to answer your question, <strong>Razor is a front-end technology that executes on the server-side runtime</strong>. It's only purpose is to generate the UI, which is the concern of the front-end.</p>],
  <p>Razor is for writing dynamic html page which is front end and c# is for writing backend logic. Although you could move all the backend logic inside razor but its highly not recommended. </p>],
  <p>Razor allows you on the <strong>back-end</strong> more easily create views (.cshtml in C#).
  It is more like templating system...</p>,
  <p><a href="http://www.asp.net/web-pages/overview/getting-started/introducing-razor-syntax-c" rel="nofollow">http://www.asp.net/web-pages/overview/getting-started/introducing-razor-syntax-c</a></p>]]
```

```
result = {}
```

```
# ใช้ for loop แบ่งข้อมูลและใส่ id
for i in range(len(answers)):
    key = f"id_{i+1}"
    result[key] = answers[i]
```

```
# แสดงผล
import pprint
pprint.pprint(result)
```

```
{'id_1': [<p>It is not a framework . I think you're misinterpreting certain concepts. Razor is a server side view engine, and it
  <p><a href="http://haacked.com/archive/2011/01/06/razor-syntax-quick-reference.aspx/" rel="noreferrer">Razor Syntax Quick Reference</a></p>],
  'id_2': [<p>This question is a couple of years old, but I'm going to add my two cents.</p>,
  <p>People struggle to give an answer to this question because the terms 'front-end' and 'back-end' aren't formally defined anywhere. Because of that, any answer is purely subjective.</p>,
  <p>That being said, it is my <em>opinion</em> that the relationship between front-end/back-end and client-side/server-side isn't necessarily one-to-one</p>,
  <p>I think it helps to think of it like this: <strong>client-side and server-side are run-times</strong>, while <strong>front-end and back-end are a separation of concerns</strong>.</p>,
  <p>'Client-side' always refers to code executing on the client's machine and 'server-side' always refers to code executing on the server. A 'front-end' developer deals with displaying data to the user and getting data from the user, while a back-end developer deals with storing, manipulating, and retrieving that data.</p>,
  <p>Consider a front-end developer who is tasked with building a UI. Much of the code they write will be the typical HTML / CSS / JS. However, they will also have to deal with the data that is passed to the front-end from the backend. This is where Razor comes into play. The front-end developer will write the Razor code (which executes on the server-side) to display the data. </p>,
  <p>That is, <strong>the front-end developer will write server-side code to help generate the UI, in addition to writing the client-side code that really defines the UI</strong>.</p>,
  <p>Now, I can't imagine a scenario where a back-end developer will write client-side code.</p>,
  <p>So, to answer your question, <strong>Razor is a front-end technology that executes on the server-side runtime</strong>. It's only purpose is to generate the UI, which is the concern of the front-end.</p>],
  'id_3': [<p>Razor is for writing dynamic html page which is front end and c# is for writing backend logic. Although you could move all the backend logic inside razor but its highly not recommended. </p>],
  'id_4': [<p>Razor allows you on the <strong>back-end</strong> more easily create views (.cshtml in C#).
  It is more like templating system...</p>,
  <p><a href="http://www.asp.net/web-pages/overview/getting-started/introducing-razor-syntax-c" rel="nofollow">http://www.asp.net/web-pages/overview/getting-started/introducing-razor-syntax-c</a></p>]]
```

## 1.cleaning

```
# remove html tags
def clean_html_tags(text):
    clean_text = text.get_text(separator=' ')
    clean_text = re.sub(r'\s+', ' ', clean_text).strip()

    return clean_text
```

```
def masking(text):
    return re.sub(r'razor', '[framework]', text)

stop_words = set(stopwords.words('english'))
preprocessed_result = {}

for key, contents in result.items():
    preprocessed_result[key] = []
    for text in contents:
        # remove html tags
        cleaned_text = clean_html_tags(text)
        # lowercase
        question = cleaned_text.lower()
        # tokenize
        words = word_tokenize(question)
```

```

# remove stopwords
filtered_words = [word for word in words if word not in stop_words]
# join words
question_cleaning_text = " ".join(filtered_words)
# remove special characters
question_cleaning_text = re.sub(r'[^a-zA-Z0-9\s]', '', question_cleaning_text)
question_cleaning_text = re.sub(r'\s+', ' ', question_cleaning_text).strip()
# data masking
question_cleaning_text = masking(question_cleaning_text)
# append to result
preprocessed_result[key].append(question_cleaning_text)

pprint.pprint(preprocessed_result)

{'id_1': ['framework think re misinterpreting certain concepts [framework] '
          'server side view engine uses c vbnet generate dynamic content',
          '[framework] syntax quick reference'],
 'id_2': ['question couple years old m going add two cents',
          'people struggle give answer question terms frontend backend nt '
          'formally defined anywhere answer purely subjective',
          'said opinion relationship frontendbackend clientsideserverside nt '
          'necessarily onetoone',
          'think helps think like clientside serverside runtimes frontend '
          'backend separation concerns',
          'clientside always refers code executing client s machine serverside '
          'always refers code executing server frontend developer deals '
          'displaying data user getting data user backend developer deals '
          'storing manipulating retrieving data',
          'consider frontend developer tasked building ui much code write '
          'typical html css js however also deal data passed frontend backend '
          '[framework] comes play frontend developer write [framwork] code '
          'executes serverside display data',
          'frontend developer write serverside code help generate ui addition '
          'writing clientside code really defines ui',
          'ca nt imagine scenario backend developer write clientside code',
          'answer question [framework] frontend technology executes serverside '
          'runtime s purpose generate ui concern frontend'],
 'id_3': [['framework] writing dynamic html page front end c writing backend '
          'logic although could move backend logic inside [framework] highly '
          'recommended'],
 'id_4': [['framework] allows backend easily create views cshtml c like '
          'templating system',
          'http '
          'wwwaspnetwebpagesoverviewgettingstartedintroducing[framework]syntaxc'}}

```

## 2. Pre-Processing

```

for key, contents in preprocessed_result.items():
    print(f"\n--- Processing {key} ---")
    for i, text in enumerate(contents):
        # Lemmatization สำหรับ noun
        words = text.split(' ')
        lemmatized_n = [lemmatizer.lemmatize(word, pos='n') for word in words]
        # Lemmatization สำหรับ verb
        lemmatized_nv = [lemmatizer.lemmatize(word, pos='v') for word in lemmatized_n]
        question_preprocessing_text = ' '.join(lemmatized_nv)
        print(f"\nOriginal Text: {text}")
        print(f"Leemmatized Text: {question_preprocessing_text}")

        # POS tagging
        tokens_for_pos = word_tokenize(question_preprocessing_text)
        pos_tags = pos_tag(tokens_for_pos)
        print(f"POS Tags: {pos_tags}")

```

--- Processing id\_1 ---

Original Text: framework think re misinterpreting certain concepts [framework] server side view engine uses c vbnet generate d  
 Lemmatized Text: framework think re misinterpret certain concept [framework] server side view engine us c vbnet generate dynam  
 POS Tags: [('framework', 'NN'), ('think', 'VBP'), ('re', 'NN'), ('misinterpret', 'NN'), ('certain', 'JJ'), ('concept', 'NN'),

Original Text: [framework] syntax quick reference  
 Lemmatized Text: [framework] syntax quick reference  
 POS Tags: [('['', 'JJ'), ('framework', 'NN'), (']', 'NNP'), ('syntax', 'NN'), ('quick', 'NN'), ('reference', 'NN')]

--- Processing id\_2 ---

Original Text: question couple years old m going add two cents  
 Lemmatized Text: question couple year old m go add two cent

POS Tags: [('question', 'NN'), ('couple', 'NN'), ('year', 'NN'), ('old', 'JJ'), ('m', 'NN'), ('go', 'VBP'), ('add', 'RB'), ('

Original Text: people struggle give answer question terms frontend backend nt formally defined anywhere answer purely subject  
 Lemmatized Text: people struggle give answer question term frontend backend nt formally define anywhere answer purely subject  
 POS Tags: [('people', 'NNS'), ('struggle', 'VBP'), ('give', 'JJ'), ('answer', 'JJR'), ('question', 'NN'), ('term', 'NN'), ('f

Original Text: said opinion relationship frontendbackend clientsideserverside nt necessarily onetoone  
 Lemmatized Text: say opinion relationship frontendbackend clientsideserverside nt necessarily onetoone  
 POS Tags: [('say', 'VB'), ('opinion', 'NN'), ('relationship', 'NN'), ('frontendbackend', 'VBP'), ('clientsideserverside', 'NN')]

Original Text: think helps think like clientside serverside runtimes frontend backend separation concerns  
 Lemmatized Text: think help think like clientside serverside runtimes frontend backend separation concern  
 POS Tags: [('think', 'VB'), ('help', 'NN'), ('think', 'VB'), ('like', 'IN'), ('clientside', 'NN'), ('serverside', 'NN'), ('runtimes', 'NN'), ('frontend', 'NN'), ('backend', 'NN'), ('separation', 'NN'), ('concerns', 'NN')]

Original Text: clientside always refers code executing client s machine serverside always refers code executing server frontend  
 Lemmatized Text: clientside always refer code execute client s machine serverside always refer code execute server frontend  
 POS Tags: [('clientside', 'NN'), ('always', 'RB'), ('refer', 'VBP'), ('code', 'JJ'), ('execute', 'JJ'), ('client', 'NN'), ('s', 'NN'), ('machine', 'NN'), ('serverside', 'NN'), ('always', 'RB'), ('refer', 'VBP'), ('code', 'JJ'), ('execute', 'JJ'), ('server', 'NN'), ('frontend', 'NN')]

Original Text: consider frontend developer tasked building ui much code write typical html css js however also deal data pass  
 Lemmatized Text: consider frontend developer task build ui much code write typical html cs j however also deal data pass from  
 POS Tags: [('consider', 'VB'), ('frontend', 'JJ'), ('developer', 'NN'), ('task', 'NN'), ('build', 'VB'), ('ui', 'JJ'), ('much', 'NN'), ('code', 'NN'), ('write', 'VB'), ('typical', 'JJ'), ('html', 'NN'), ('css', 'NN'), ('js', 'NN'), ('however', 'RB'), ('also', 'RB'), ('deal', 'VB'), ('data', 'NN'), ('pass', 'NN'), ('from', 'IN')]

Original Text: frontend developer write serverside code help generate ui addition writing clientside code really defines ui  
 Lemmatized Text: frontend developer write serverside code help generate ui addition write clientside code really define ui  
 POS Tags: [('frontend', 'NN'), ('developer', 'NN'), ('write', 'NN'), ('serverside', 'NN'), ('code', 'NN'), ('help', 'NN'), ('addition', 'NN'), ('write', 'NN'), ('clientside', 'NN'), ('code', 'NN'), ('really', 'RB'), ('define', 'NN'), ('ui', 'NN')]

Original Text: ca nt imagine scenario backend developer write clientside code  
 Lemmatized Text: ca nt imagine scenario backend developer write clientside code  
 POS Tags: [('ca', 'MD'), ('nt', 'RB'), ('imagine', 'VB'), ('scenario', 'NN'), ('backend', 'NN'), ('developer', 'NN'), ('write', 'NN'), ('clientside', 'NN'), ('code', 'NN')]

Original Text: answer question [framework] frontend technology executes serverside runtime s purpose generate ui concern front  
 Lemmatized Text: answer question [framework] frontend technology execute serverside runtime s purpose generate ui concern from  
 POS Tags: [('answer', 'JJR'), ('question', 'NN'), ('[', 'NNP'), ('framework', 'NN'), (']', 'NNP'), ('frontend', 'NN'), ('technology', 'NN'), ('executes', 'NN'), ('serverside', 'NN'), ('runtime', 'NN'), ('s', 'NN'), ('purpose', 'NN'), ('generate', 'NN'), ('ui', 'NN'), ('concern', 'NN'), ('front', 'NN')]

--- Processing id\_3 ---

Original Text: [framework] writing dynamic html page front end c writing backend logic although could move backend logic inside  
 Lemmatized Text: [framework] write dynamic html page front end c write backend logic although could move backend logic inside  
 POS Tags: [('framework', 'NN'), ('writing', 'NN'), ('dynamic', 'JJ'), ('html', 'NN'), ('page', 'NN'), ('front', 'NN'), ('end', 'NN'), ('c', 'NN'), ('writing', 'NN'), ('backend', 'NN'), ('logic', 'NN'), ('although', 'RB'), ('could', 'MD'), ('move', 'VB'), ('backend', 'NN'), ('logic', 'NN'), ('inside', 'IN')]

--- Processing id\_4 ---

Punchaya Chancharoen 65070507236